2.2.2 Using Index Files

Sprockets uses files named `index` (with the relevant extensions) for a special purpose.
For example, if you have a jQuery library with many modules, which is stored in `lib/assets/javascripts/library_name`, the file `lib/assets/javascripts/library_name/index.js` serves as the manifest for all files in this library. This file could include a list of all the required files in order, or a simple `require_tree`directive.
The library as a whole can be accessed in the application manifest like so:

```
//= require library_name
```

This simplifies maintenance and keeps things clean by allowing related code to be grouped before inclusion elsewhere.

2.3 Coding Links to Assets

Sprockets does not add any new methods to access your assets - you still use the familiar `javascript_include_tag` and `stylesheet_link_tag`:

```
<%= stylesheet_link_tag "application", media: "all" %>
<%= javascript_include_tag "application" %>
```

If using the turbolinks gem, which is included by default in Rails 4, then include the 'data-turbolinks-track' option which causes turbolinks to check if an asset has been updated and if so loads it into the page:

```
<%= stylesheet_link_tag "application", media: "all", "data-turbolinks-
track" => true %>
<%= javascript_include_tag "application", "data-turbolinks-track" =>
true %>
```

In regular views you can access images in the `public/assets/images` directory like this:

```
<%= image_tag "rails.png" %>
```

Provided that the pipeline is enabled within your application (and not disabled in the current environment context), this file is served by Sprockets. If a file exists at `public/assets/rails.png` it is served by the web server.
Alternatively, a request for a file with an MD5 hash such as `public/assets/rails-af27b6a414e6da00003503148be9b409.png` is treated the same way. How these hashes are generated is covered in the In Production section later on in this guide.
Sprockets will also look through the paths specified in `config.assets.paths`, which includes the standard application paths and any paths added by Rails engines.
Images can also be organized into subdirectories if required, and then can be accessed by specifying the directory's name in the tag:

```
<%= image_tag "icons/rails.png" %>
```

If you're precompiling your assets (see In Production below), linking to an asset that does not exist will raise an exception in the calling page. This includes linking to a blank string. As such, be careful using `image_tag` and the other helpers with user-supplied data.

2.3.1 CSS and ERB

The asset pipeline automatically evaluates ERB. This means if you add an `erb` extension to a CSS asset (for example, `application.css.erb`), then helpers like `asset_path` are available in your CSS rules:

```
.class { background-image: url(<%= asset_path 'image.png' %>) }
```

This writes the path to the particular asset being referenced. In this example, it would make sense to have an image in one of the asset load paths, such as `app/assets/images/image.png`, which would be